Listing 4.10 `autoget_coding_group.py` (relevant sections)

```
user_proxy = UserProxyAgent(
    "user",
    code_execution_config={
        "work_dir": "working",
        "use_docker": False,
        "last_n_messages": 3,
    },
    human_input_mode="NEVER",     #1
)

llm_config = {"config_list": config_list}

engineer = AssistantAgent(…      #2


critic = AssistantAgent(…       #2


groupchat = GroupChat(agents=[user_proxy,
                              engineer,
                              critic],
                              messages=[],
                              max_round=20)      #3
manager = GroupChatManager(groupchat=groupchat,
                           llm_config=llm_config)     #4

task = """Write a snake game using Pygame."""

with Cache.disk(cache_seed=43) as cache:
    res = user_proxy.initiate_chat(
        recipient=manager,
        message=task,
        cache=cache,
    )
```

#1 Human input is now set to never, so no human feedback.
#2 Code omitted, but consult changes to the persona in the file
#3 This object holds the connection to all the agents and stores the messages.
#4 The manager coordinates the conversation as a moderator would.

Run this exercise, and you'll see how the agents collaborate. The engineer will now take feedback from the critic and undertake operations to address the critic's suggestions. This also allows the proxy to engage in all of the conversation.

Group conversations are an excellent way to strengthen your agents' abilities as they collaborate on tasks. However, they are also substantially more verbose and token expensive. Of course, as LLMs mature, so do the size of their context token windows and the price of token processing. As token windows increase, concerns over token consumption may eventually go away.